

Backblaze SMART HDD Predictive Maintenance System

using Deep Learning and Temporal Sequence Modeling

Feras Afi

EPI Digital School

Course: Stochastic Models

May 12, 2026

Contents

1	Introduction to S.M.A.R.T. Technology	5
2	The Backblaze Dataset	5
2.1	Contained Information	6
2.2	Definition of Failure	6
3	Methodology	6
3.1	Data Cleaning Pipeline	6
3.1.1	Loading Data	6
3.1.2	Removing Constant Columns	7
3.1.3	Handling Missing Values	7
3.1.4	Removing Duplicates	7
3.1.5	Exporting Cleaned Data	7
3.1.6	Results Summary	8
3.2	Exploratory Data Analysis (EDA)	8
3.2.1	Dataset Overview	8
3.2.2	Drive and Model Distribution	8
3.2.3	Class Imbalance	9
3.2.4	Capacity Analysis	9
3.2.5	Temporal Failure Pattern	10
3.3	Preprocessing	10
3.3.1	Column Alignment	10
3.3.2	Date Sorting	10
3.3.3	Feature Selection	10
3.3.4	Failed Drive Extraction	11
3.3.5	Balancing Normal and Failed Drives	11
3.3.6	Handling Missing Days	11
3.3.7	Normalization	12
3.3.8	Sequence Creation	12
3.3.9	Final Dataset Shapes	12
4	Modeling	12
4.1	Data Preparation for Sequence Models	12
4.2	Model Architectures	13
4.2.1	CNN-LSTM Architecture	13
4.2.2	TCN Architecture (Temporal Convolutional Network)	14

4.2.3	LSTM + Attention Architecture	16
5	Evaluation	17
5.1	CNN-LSTM Results	17
5.2	LSTM + Attention Results	17
5.3	TCN Results	19
5.4	Comparison and Conclusion	19

General Objective

The objective of this lab is to design and implement a predictive maintenance system capable of anticipating equipment failure from the analysis of industrial sensor time series. In real-world scenarios, machines and embedded systems continuously generate signals that reflect their operating state. The challenge is to exploit these temporal data to detect early warning signs of failure before they occur.

Unlike conventional approaches that treat each observation independently, this lab relies on sequential modeling of time series. We will study and compare three neural architectures:

- **CNN–LSTM**: A hybrid model where convolutional layers extract local patterns (e.g., short anomalies or trend changes) and LSTM layers capture long-term temporal dependencies.
- **Temporal Convolutional Network (TCN)**: A fully convolutional architecture that uses dilated causal convolutions to achieve a large receptive field while avoiding recurrent units. It is trained in parallel and often more stable than LSTMs.
- **LSTM + Attention**: A recurrent model augmented with an attention mechanism that dynamically weighs the importance of different time steps, allowing the network to focus on the most failure-relevant periods in the sequence.

Key Idea

All three approaches combine local signal analysis with temporal memory, but through different principles: convolution for local features (CNN-LSTM, TCN) or recurrent gates with selective focus (LSTM+Attention). This combination enables more effective detection of failure-indicating patterns compared to traditional threshold-based or single-observation classifiers.

1 Introduction to S.M.A.R.T. Technology

Modern hard drives are equipped with an internal mechanism called S.M.A.R.T. (Self-Monitoring, Analysis and Reporting Technology). This system automatically records drive health indicators such as: the number of internal errors, temperature, total operating time, and the number of reallocated bad sectors. These indicators provide a temporal trace of drive behavior and make it possible to progressively detect signs of degradation.



Figure 1: Backblaze HDDs

2 The Backblaze Dataset

Backblaze is a company specialized in large-scale cloud storage. It operates real data centers containing hundreds of thousands of production hard drives. In a spirit of scientific transparency, it provides the public with an open dataset containing S.M.A.R.T. readings from its drives. This dataset is the main support used in this project.

The Backblaze Hard Drive SMART dataset (2017 edition) is a real time-series dataset collected daily under industrial conditions. It makes it possible to track the evolution of drive health over long periods. In general: each row represents a hard drive on a given date, the data are recorded daily, and monitoring spans several years.

2.1 Contained Information

The dataset contains several categories of variables describing the state of hard drives:

- **Date:** indicates the day when SMART data were collected. It enables ordering observations and analyzing drive evolution over time.
- **Serial Number:** the unique identifier for each drive, allowing tracking of the same drive over multiple days.
- **Model:** the manufacturer reference, useful for comparing reliability across different models.
- **Capacity:** the total size of the drive expressed in bytes.
- **Failure Indicator:** a binary variable (0: healthy, 1: failed). This is the target variable for supervised learning.
- **SMART Attributes:** a set of internal indicators describing drive behavior (errors, temperature, delays, mechanical instabilities, etc.).

2.2 Definition of Failure

A drive is considered failed when it stops sending SMART data. This information serves as the label for supervised learning. Note that failures are rare compared with normal states. The dataset is therefore highly imbalanced, as in real industrial environments. This characteristic justifies the use of sequential models.

3 Methodology

3.1 Data Cleaning Pipeline

The data cleaning process follows a systematic pipeline applied to both training (Q4 2016) and testing (Q3 2017) datasets.

3.1.1 Loading Data

The `load_data` function reads all CSV files from a given directory, removes columns containing the word "normalized", and concatenates them into a single DataFrame.

3.1.2 Removing Constant Columns

Columns with zero variance (only one distinct value or identical min/max) provide no predictive information and are removed. This step reduces dimensionality without information loss.

3.1.3 Handling Missing Values

Three strategies are applied:

- **Columns:** Columns with missing values above a defined threshold (default 100%) are completely dropped.
- **Rows:** Rows missing critical identifiers (`date`, `serial_number`, `model`, `capacity_bytes`, `failure`) are removed.
- **SMART attributes:** Remaining missing values in SMART columns are filled with 0, assuming absence of the measured phenomenon.

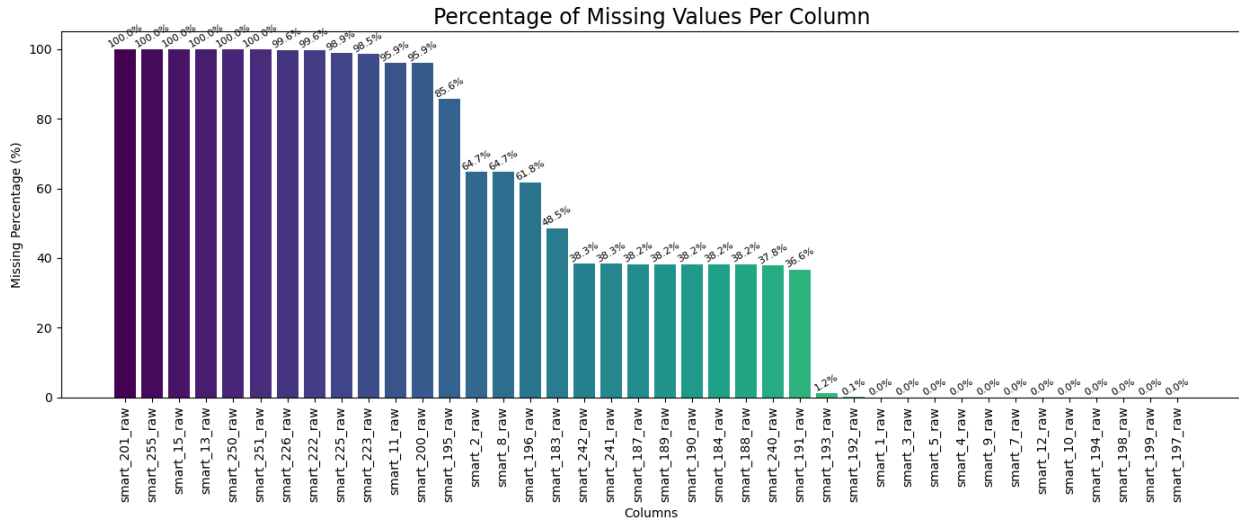


Figure 2: Percentage of missing values per column before imputation

3.1.4 Removing Duplicates

Duplicate rows are identified and removed to avoid data leakage and over-representation.

3.1.5 Exporting Cleaned Data

Final cleaned DataFrames are exported as Pickle files for efficient storage and fast loading in subsequent steps.

3.1.6 Results Summary

- **Training set (Q4 2016):** Final shape = (6589448, 41)
- **Testing set (Q3 2017):** Final shape = (8004840, 42)
- **Constant columns removed:** 'smart₂20', 'smart₂24', 'smart₂52', 'smart₂54'
- **Columns with high missingness dropped:** 4
- **Duplicates removed:** 0 trainset, 0 testset

3.2 Exploratory Data Analysis (EDA)

3.2.1 Dataset Overview

The cleaned training dataset contains 6589448 rows and 41 columns. The test dataset contains 8004840 rows and 42 columns.

3.2.2 Drive and Model Distribution

The training set comprises 74453 unique hard drives across 59 distinct models. As shown in Figure 3, the most frequent serial numbers reveal drives that appear repeatedly throughout the dataset.

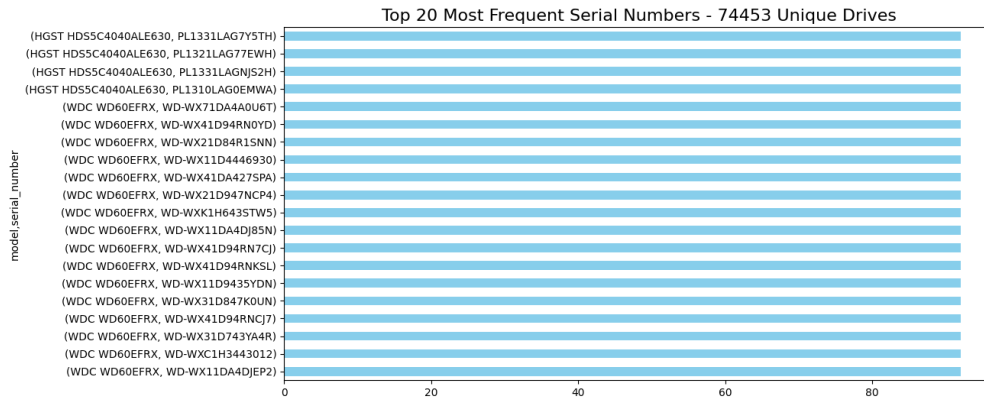


Figure 3: Top 20 most frequent serial numbers

Regarding drive models, Figure 4 displays the most common manufacturer references present in the dataset.

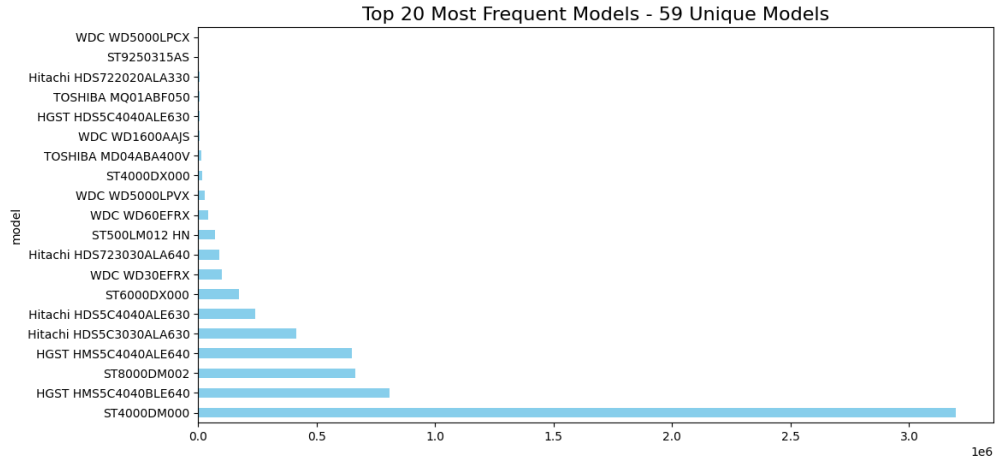


Figure 4: Top 20 most frequent models

3.2.3 Class Imbalance

The failure indicator shows a highly imbalanced distribution: healthy drives represent 99.9947% of the dataset, while failed drives account for only 0.0053%. This severe imbalance justifies the use of sequential models and appropriate evaluation metrics such as precision, recall, and F1-score rather than raw accuracy.

3.2.4 Capacity Analysis

Drive capacity ranges from 160 GB to 8 TB, with a mean of 4.3 TB. Figure 5 presents the overall capacity distribution. Comparing normal versus failed drives, no significant capacity-based pattern emerges, suggesting that failure is not strongly correlated with storage size.

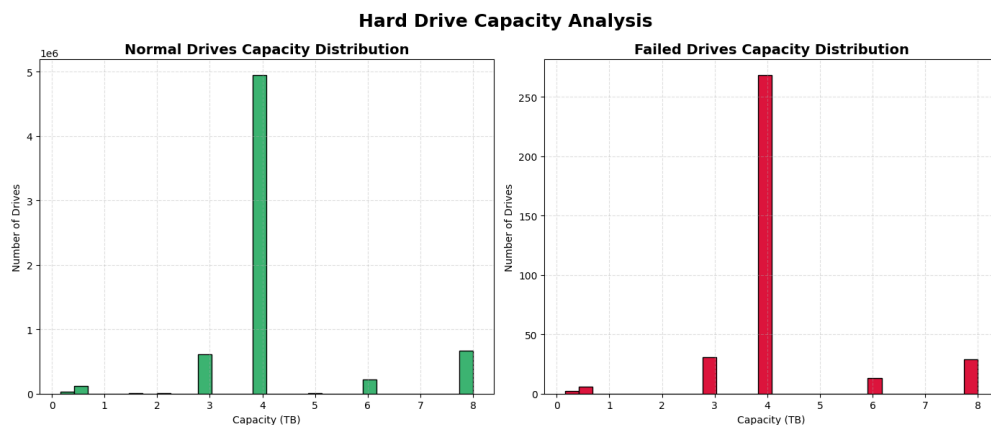


Figure 5: Hard drive capacity distribution (all drives)

3.2.5 Temporal Failure Pattern

Failures are not uniformly distributed over time. Figure 6 highlights the dates with the highest failure concentrations, which may correspond to specific production batches or environmental conditions.

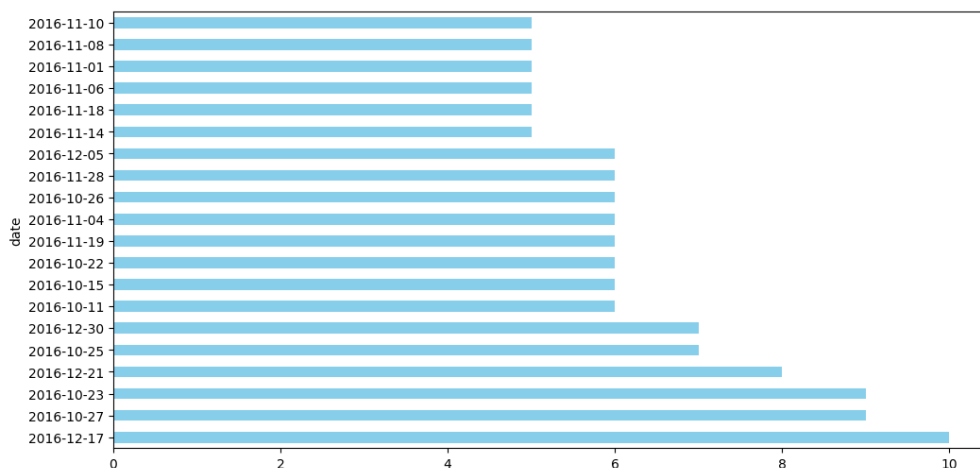


Figure 6: Top 20 dates with highest failure counts

3.3 Preprocessing

3.3.1 Column Alignment

The training set has shape (6589448, 41) and the test set (8004840, 42). After identifying mismatched columns, `smart_22_raw` (present only in test) was removed. Both datasets were then aligned to use identical column sets.

3.3.2 Date Sorting

The `date` column was converted to datetime format and both DataFrames were sorted chronologically to preserve temporal order.

3.3.3 Feature Selection

Correlation analysis with the failure target was performed. A Random Forest classifier was trained on numerical features to compute feature importances. The top 28 SMART attributes showing the highest importance scores were selected, alongside all metadata columns (`date`, `serial_number`, `model`, `capacity_bytes`, `failure`).

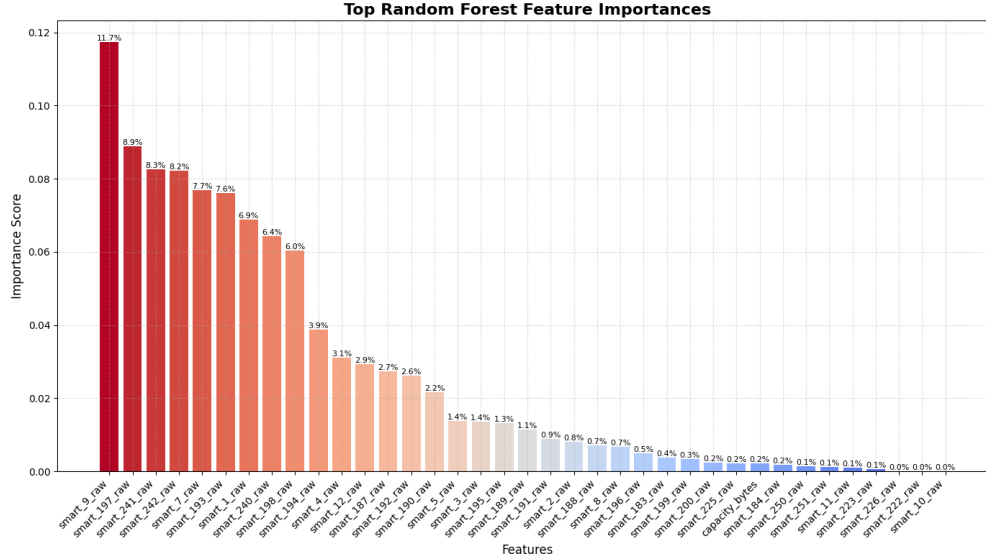


Figure 7: Feature Selection via Random Forest Classifier

3.3.4 Failed Drive Extraction

Failed drives were isolated for further analysis. Figure 7 shows the normalized multi-sensor evolution of a sample drive leading to failure.

3.3.5 Balancing Normal and Failed Drives

The raw dataset exhibits severe class imbalance, a common characteristic of real-world predictive maintenance applications. Failed drives represent only 99.9947% of all observations, while healthy drives dominate the remaining 0.0053%. This imbalance, illustrated in Figure 8, would cause any model trained on raw data to bias predictions toward the majority class, achieving high accuracy but failing to detect failures.

To mitigate this issue, a balanced sampling strategy was adopted. All failed drives (392 drives) were retained to preserve all failure patterns. For healthy drives, only 500 randomly selected drives were kept. This yields a near-balanced training set with 392 failed and 500 healthy drives, preventing the model from ignoring the minority failure class while maintaining computational efficiency. The test set follows the same sampling strategy to ensure consistent evaluation.

3.3.6 Handling Missing Days

For each drive, missing dates were filled using forward fill to create continuous daily time series. This ensures regular intervals for sequence modeling.

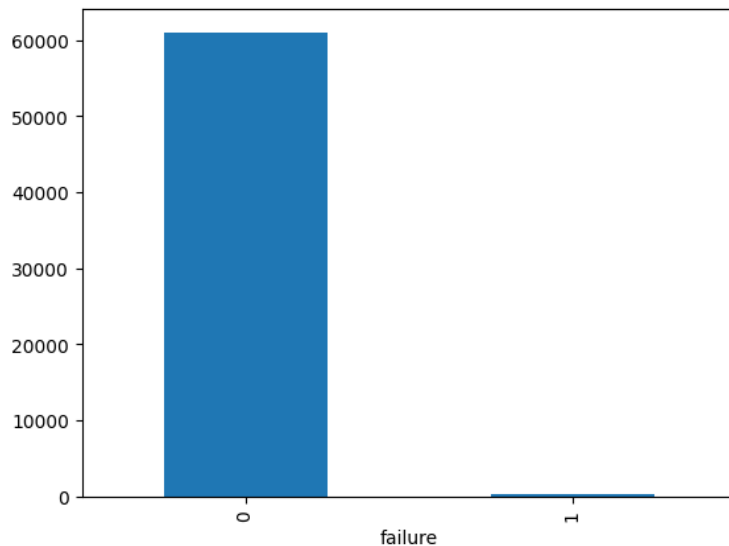


Figure 8: Distribution of healthy vs failed drives before balancing

3.3.7 Normalization

MinMaxScaler was applied to all SMART attribute columns, scaling values to the range $[0, 1]$. The scaler was fitted on training data and used to transform both training and test sets.

3.3.8 Sequence Creation

Using a sliding window approach, sequences of length 5 days were created. For failed drives, each sequence ends one day before failure (lookahead = 1 day). For healthy drives, all consecutive 5-day windows were extracted. Each sequence is labeled 1 (will fail) or 0 (will not fail).

3.3.9 Final Dataset Shapes

- Training sequences: (281875, 5, 28) samples
- Test sequences: (287225, 5, 28) samples

Both sequence sets were exported as Pickle files for model training.

4 Modeling

4.1 Data Preparation for Sequence Models

Before training, the sequence data must be reshaped into the format expected by neural networks. The raw sequences contain 5 time steps (days) with 28 features per step. Using

a sliding window approach with stride 1, the final input tensors have shape (samples, 5, features). The target variable is the label of the last day in each window.

The training set contains (281875, 5, 28) samples and the test set (287225, 5, 28) samples.

4.2 Model Architectures

Three distinct neural architectures are implemented and compared for predictive maintenance.

4.2.1 CNN-LSTM Architecture

The CNN-LSTM hybrid combines convolutional layers for local pattern extraction with LSTM layers for temporal dependency modeling. The architecture consists of:

- Two Conv1D layers with 64 filters each, kernel size 2, ReLU activation, and same padding. These extract local sensor patterns across the 5-day window.
- A MaxPooling1D layer (pool size 2) to reduce dimensionality and retain dominant features.
- Two stacked LSTM layers with 64 units each. The first returns sequences, the second returns only the final output. Dropout (0.3) and recurrent dropout (0.2) are applied for regularization.
- A Dense layer with 64 units, tanh activation, and L2 regularization ($\lambda = 0.001$).
- A Dropout layer (0.3) for additional regularization.
- A final Dense layer with sigmoid activation for binary classification (failure vs healthy).

Layer (type)	Output Shape	Param #
Conv1D	(5, 64)	3,648
Conv1D_1	(5, 64)	8,256
MaxPooling1D	(2, 64)	0
LSTM	(5, 64)	33,024
LSTM_1	(64)	33,024
Dense	(64)	4,160
Dropout	(64)	0
Dense_1	(1)	65

Table 1: CNN-LSTM model architecture summary

Total trainable parameters: 82,177.

The model is compiled with the Adam optimizer, binary cross-entropy loss, and accuracy as the evaluation metric.

Class Weighting

Due to the remaining class imbalance in the sequence dataset, class weights are applied during training. A weight of 50 is assigned to the failure class (1) and 1 to the healthy class (0), forcing the model to pay more attention to failure samples.

Training

The model is trained for 10 epochs with a batch size of 64. Validation is performed on the test set. Figure 11 shows the training and validation accuracy and loss over epochs.

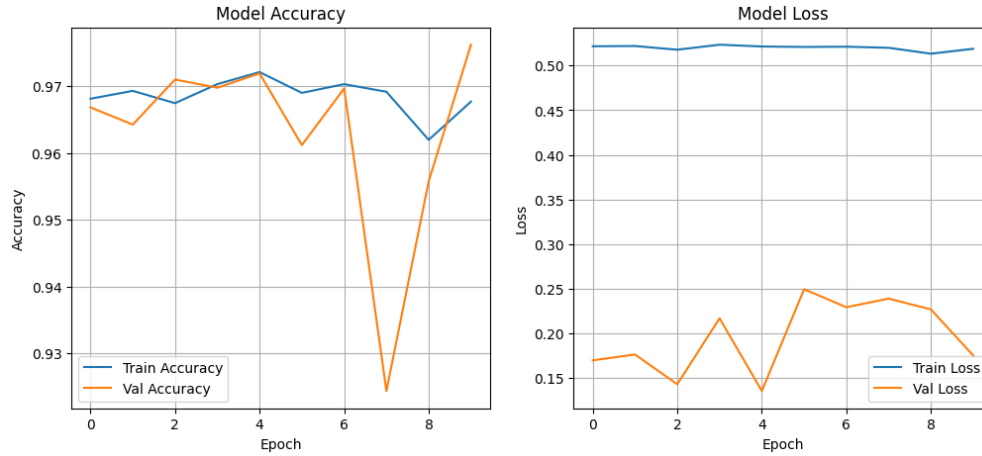


Figure 9: Training and validation accuracy (left) and loss (right) over epochs

4.2.2 TCN Architecture (Temporal Convolutional Network)

The Temporal Convolutional Network (TCN) offers a fully convolutional alternative to recurrent architectures. Unlike LSTMs that process sequences sequentially, TCN uses dilated causal convolutions to capture long-range dependencies in parallel, enabling faster training and often more stable gradients. The architecture consists of:

- A TCN layer with 64 filters, kernel size of 3, and dilations [1, 2, 4, 8]. This exponentially increasing dilation rate gives the network a wide receptive field, allowing it to capture patterns across the entire 5-day window without increasing the number of parameters.
- Dropout of 0.3 within the TCN layer for regularization.

- A first Dense layer with 64 units, ReLU activation, and L2 regularization ($\lambda = 0.001$).
- A Dropout layer (0.3) for additional regularization.
- A second Dense layer with 32 units, ReLU activation, and L2 regularization ($\lambda = 0.001$).
- A second Dropout layer (0.3).
- A final Dense layer with sigmoid activation for binary classification.

Layer (type)	Output Shape	Param #
TCN (64 filters, kernel=3, dilations=[1,2,4,8])	(64)	[placeholder]
Dense (64, ReLU, L2)	(64)	
Dropout (0.3)	(64)	0
Dense (32, ReLU, L2)	(32)	
Dropout (0.3)	(32)	0
Dense (1, sigmoid)	(1)	

Table 2: TCN model architecture summary

The model is compiled with the Adam optimizer, binary cross-entropy loss, accuracy, and AUC as evaluation metrics.

Training

The model is compiled with the Adam optimizer, binary cross-entropy loss, and metrics including accuracy and AUC. Training runs for 10 epochs with a batch size of 128, using class weights {0: 1, 1: 50}. Validation is performed on the test set.

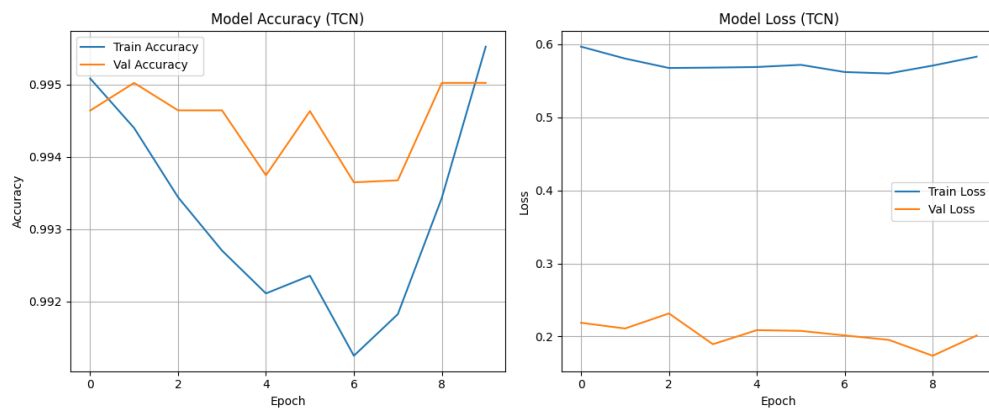


Figure 10: Training and validation accuracy (left) and loss (right) over epochs

Figure 11 shows the training and validation accuracy and loss over epochs.

4.2.3 LSTM + Attention Architecture

The LSTM with attention mechanism enhances the standard recurrent architecture by allowing the model to dynamically focus on the most relevant time steps within the input sequence. The architecture consists of:

- An Input layer accepting sequences of length 5 with `features` features.
- A first LSTM layer with 128 units, returning sequences to preserve temporal information for the attention mechanism.
- A second LSTM layer with 64 units, also returning sequences.
- An Attention layer that computes attention weights over the LSTM outputs, enabling the model to selectively focus on critical time steps preceding failure.
- A GlobalAveragePooling1D layer to aggregate the attended information.
- A Dense layer with 64 units and ReLU activation.
- A Dropout layer (0.3) for regularization.
- A final Dense layer with sigmoid activation for binary classification.

Layer (type)	Output Shape	Param #
Input	(5, features)	0
LSTM (128, return_sequences=True)	(5, 128)	
LSTM (64, return_sequences=True)	(5, 64)	
Attention	(5, 64)	
GlobalAveragePooling1D	(64)	0
Dense (64)	(64)	
Dropout (0.3)	(64)	0
Dense (1, sigmoid)	(1)	

Table 3: LSTM + Attention model architecture summary

The attention mechanism computes a context vector as a weighted sum of the LSTM hidden states, where the weights indicate each time step’s importance for the failure prediction task.

Class Weighting

Due to the remaining class imbalance in the sequence dataset, class weights are applied during training. A weight of 50 is assigned to the failure class (1) and 1 to the healthy class (0), forcing the model to pay more attention to failure samples.

Training

The model is trained for 10 epochs with a batch size of 64. Validation is performed on the test set. Figure 11 shows the training and validation accuracy and loss over epochs.

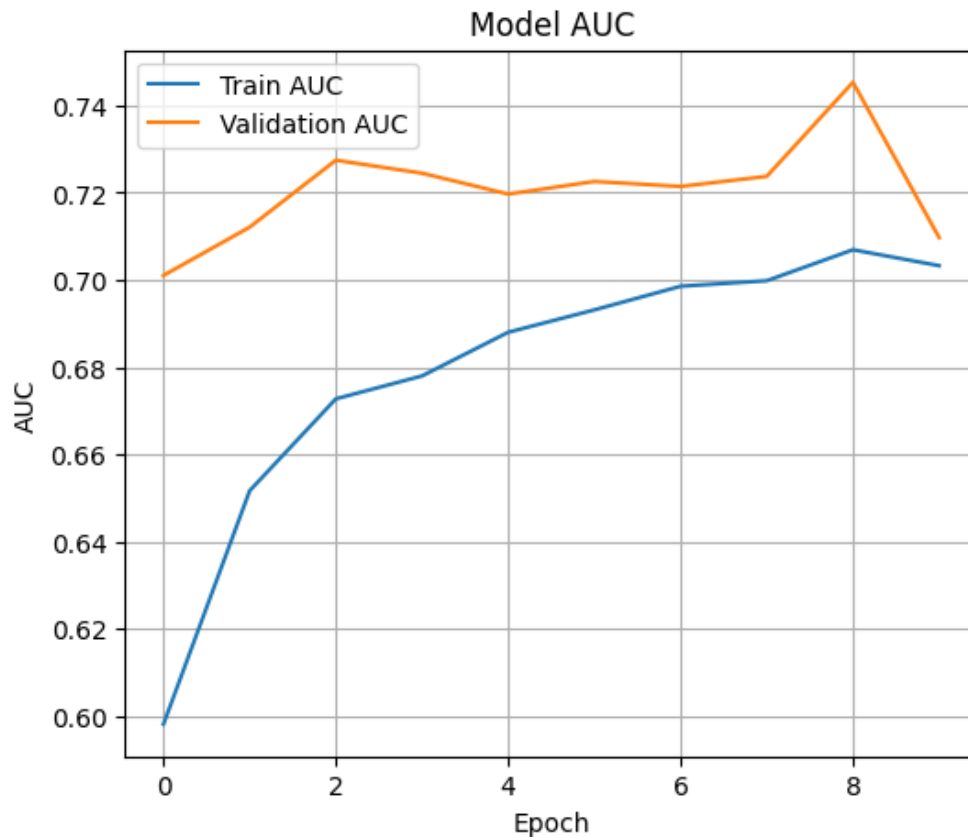


Figure 11: Training and validation accuracy (left) and loss (right) over epochs

5 Evaluation

5.1 CNN-LSTM Results

The CNN-LSTM model achieves an overall accuracy of 98%. For the failure class (1), precision is 0.04, recall is 0.17, and F1-score is 0.07.

The model correctly identifies 17% of actual failures but produces many false positives due to the class imbalance.

5.2 LSTM + Attention Results

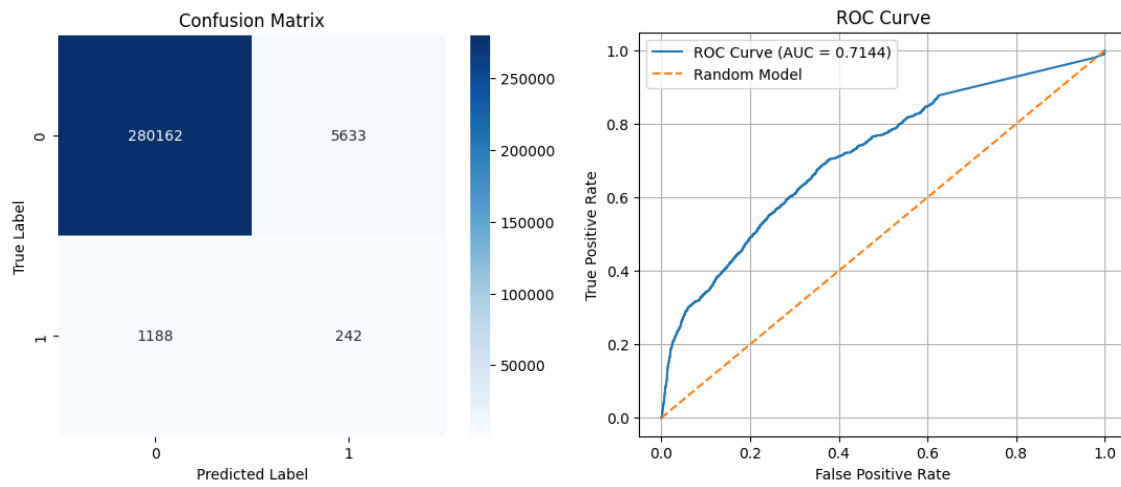


Figure 12: CNN-LSTM: Confusion matrix (left) and ROC curve (right)

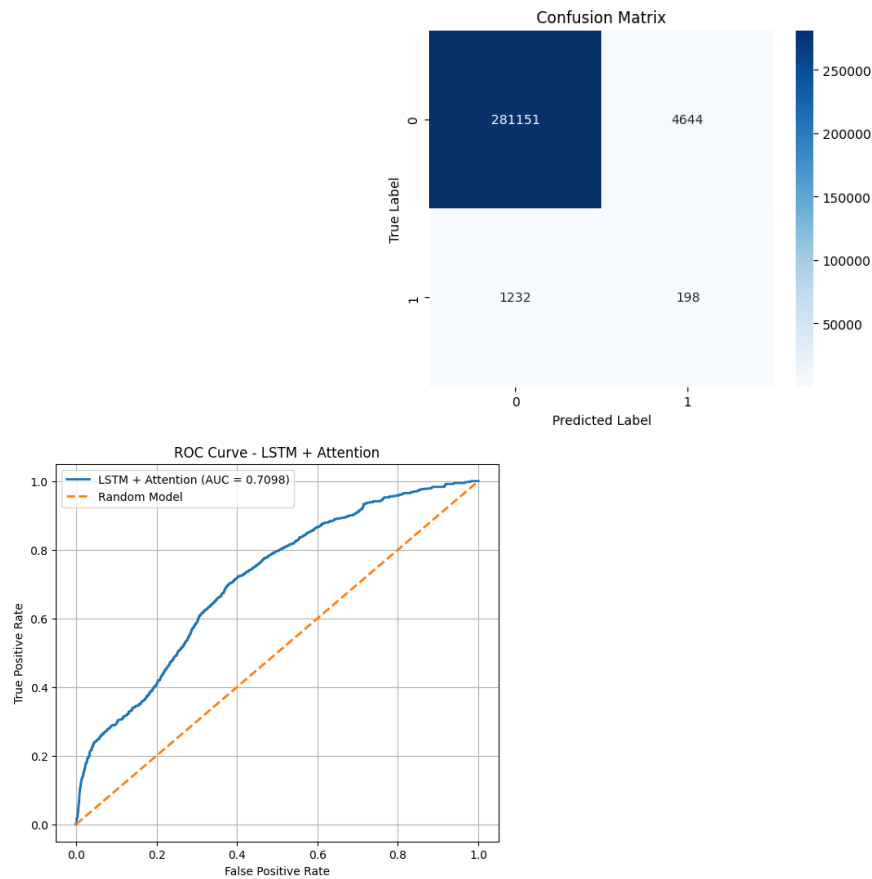


Figure 13: LSTM + Attention: Confusion matrix (left) and ROC curve (right)

The LSTM with attention mechanism achieves 98% accuracy. Failure class performance shows precision of 0.04, recall of 0.14, and F1-score of 0.06.

Attention weights provide interpretability but do not significantly improve failure detection over the baseline CNN-LSTM.

5.3 TCN Results

The TCN achieves 100% accuracy but completely fails to detect any failures. Precision, recall, and F1-score for the failure class are all 0.00. The model predicts only the majority class (healthy), demonstrating that accuracy alone is misleading for imbalanced datasets.

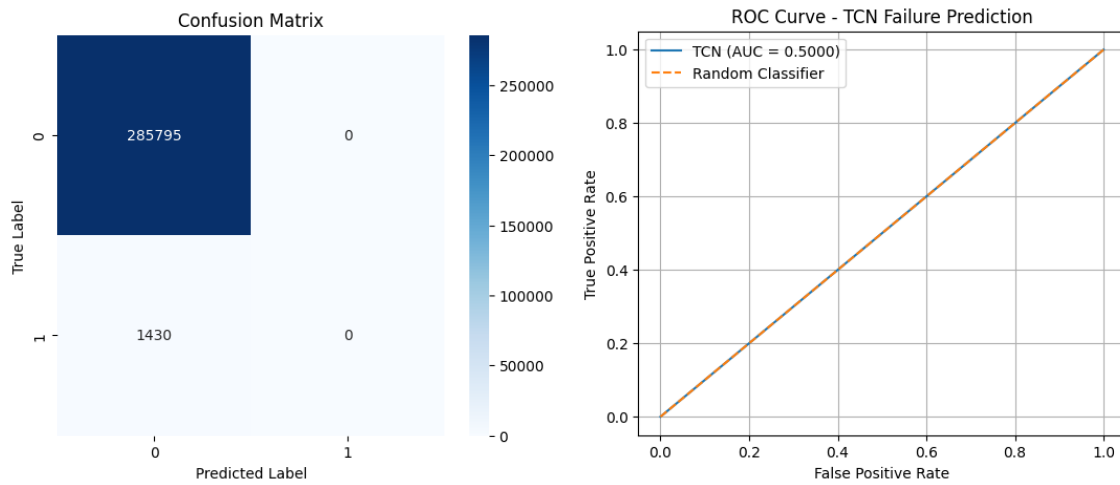


Figure 14: TCN: Confusion matrix (left) and ROC curve (right)

5.4 Comparison and Conclusion

Model	Accuracy	Precision (class 1)	Recall (class 1)	F1 (class 1)
CNN-LSTM	98%	0.04	0.17	0.07
LSTM + Attention	98%	0.04	0.14	0.06
TCN	100%	0.00	0.00	0.00

Table 4: Model performance comparison

Findings:

- All models struggle with failure detection due to extreme class imbalance (failures represent less than 0.5% of the dataset).
- CNN-LSTM achieves the best recall (17%) among the three, making it the most suitable for predictive maintenance where detecting failures is the priority.

- LSTM + Attention performs similarly but slightly worse than CNN-LSTM.
- TCN fails entirely, defaulting to predicting only healthy drives despite high accuracy.

Limitations:

- The class weight of 50 may be insufficient; higher weights or alternative sampling strategies could be explored.
- Sequence length of 5 days may be too short to capture long-term degradation patterns.
- Only 500 healthy drives were used; including more healthy samples could improve generalization.
- Real-world deployment requires handling of missing data and concept drift over time.